

FASE 1: COMPRENDE

1.- Exploración Conceptual: Cola FIFO vs Cola de Prioridad

Bloque de texto al inicio del archivo de código principal (como comentarios de encabezado) con tus respuestas propias a las tres preguntas. Prefija cada respuesta con # COMPRENDE-1:, # COMPRENDE-2:, # COMPRENDE-3:. No se entrega código en este reto — solo tu explicación escrita con tus propias palabras antes de comenzar a programar.

COMPRENDE-1:

Antes: frente apunta a A y final apunta a C.

Al hacer enqueue(X): conectas C con X (cambias el next del último).

Después: frente sigue en A y final ahora apunta a X (la nueva cola termina en X → null).

COMPRENDE-2:

En una cola circular, por el “dar la vuelta”, solo con frente y final hay estados donde los índices pueden “verse parecidos”, y eso puede hacer dudoso si puedes hacer enqueue o dequeue.

Una forma de resolverlo es guardar algo extra, por ejemplo size:

size == 0 → vacía

size == capacidad → llena

COMPRENDE-3:

FIFO atiende por orden de llegada, pero en urgencias importa la gravedad.

Ejemplo: llega A (gripa), luego B (fiebre), luego C (gangrena severa).

FIFO atendería A, luego B, luego C, y C podría empeorar (ej. mayor riesgo de amputación) por esperar.

Una cola de prioridad atiende primero a C por ser más urgente, y después a los menos graves.

Checkpoint Metacognitivo — COMPRENDE

¿Puedes explicar qué hace el Paso 4 de dequeue() en el pseudocódigo (ajustar final = null cuando la cola queda vacía) sin leer el código? Si no puedes, ¿por qué ese paso es crítico para el invariante de la estructura?

pues ese paso es critico por que con eso marcamos que la lista está vacía

¿Buscaste en la IA la explicación de la cola circular antes de leer el pseudocódigo de esta sección? Si sí, ¿coincidió con lo que encontraste aquí? Si no coincidió, ¿cuál fue la diferencia y cuál es correcta?

busque en el navegador en ves de la ia pero si, termino coincidiendo la estructura

¿Podrías implementar enqueue() ahora mismo en papel sin mirar el pseudocódigo? Si no, ¿cuál es la línea que no entiendes del todo?

aun no estoy seguro de poder hacerlo pero solo tengo que practicar mas, y si tuviese que escoger una linea que no entiendo del todo diria que es la parte donde tenemos que mover la referencia final al nuevo nodo

Fase 2: APLICA

2.- Proyecto: Sistema de Triage Hospitalario con Cola propia

Entregable único del proyecto: Un solo archivo de código en tu lenguaje elegido con todo integrado. La evidencia va dentro del mismo archivo como comentarios:

Clases Cola y ColaPrioridad completas con pruebas unitarias como bloques de código comentados al final del archivo.

Clase SistemaTriage funcional con la simulación de 5 pacientes ejecutándose correctamente.

Al menos 3 comentarios de decisión de diseño propios con el prefijo # DECISIÓN: (o // DECISIÓN:), por ejemplo: # DECISIÓN: Usé inserción ordenada en lugar de heap porque con <100 pacientes

simultáneos la diferencia es despreciable y el código es más fácil de depurar.

El checklist de VALIDA resuelto como comentarios al final del archivo con el prefijo # VALIDA:, indicando qué caso pasó o falló.

No se requiere ningún documento separado. Todo queda en el código.

```
#include <iostream>
#include <string>
using namespace std;

//aqui se guardan datos del paciente
struct paciente{
    string nombre;
    int prioridad; // la prioridad va del 1 al 4 donde 1 es la mas urgente
};

//===== Cola FIFO =====
class colaFIFO{
    private:
        struct nodo{
            paciente p; //los datos del paciente
            nodo * siguiente; //puntero hacia el siguiente nodo

            //parte constructora del nodo:recibe un paciente y deja siguiente
            //en nullptr
            nodo(const paciente& paciente) : p(paciente), siguiente(nullptr){}
        };
        nodo * frente; //apunta al primer nodo
        nodo * final; //apunta al ultimo nodo

    public:
        //parte constructora de la cola
        colaFIFO() : frente(nullptr), final(nullptr){}

        bool estavacia() const{
            return frente == nullptr;
        }

        //===== parte de enqueue(encolar) =====
        void enqueue(const paciente& paciente) {
```

```

//1. crear un nodo en heap
nodo * nuevo = new nodo(paciente);

//2. caso especial: si esta la cola vacia, el nuevo nodo es el primero
y el ultimo
if(estavacia()){
    frente = final = nuevo;
} else{ // si no es así el nodo final apunta a nuevo
    final -> siguiente = nuevo;
    final=nuevo; // y pasa a convertirse en nuevo
}
}

//parte de verfrente() y evidentemente es para consultar al paciente de
enfrente
paciente verfrente() const{
    //si no hay nodos no tenemos nada que devolver
    if (estavacia()){
        cout<< "cola vacia, por lo que no hay pacientes al frente";

    }

    //si no retornamos los datos del primer nodo
    return frente -> p;
}

//===== parte dequeue =====

paciente dequeue(){
    //1. validamos que la cola no este vacia
    if(estavacia()){
        cout<<"cola vacia, por lo que no podemos hacer dequeue";
    }

    //2. guardamos el nodo del frente en una variable temporal
    nodo * temp = frente; /*temp de temporal*/

    //3. guardamos al paciente para regresarlo al final del metodo
    paciente salida = frente -> p;

    //4.avanzamos frente al siguiente nodo
    frente =frente -> siguiente;

```

```

//5. si despues de avanzar la el puntero de frente queda null
significa que la cola queda vacia por lo que final tambien debe de ser
null

    if (frente == nullptr)
        final = nullptr;

//6. en esta parte liberamos el nodo que sacamos para no tener fuga
de memoria
    delete temp;

//7. y por ultimo en dequeue devolvemos al paciente que sacamos
    return salida;
}
};

// ===== cola de prioridad (sistema triage) =====
class colaprioridad{
    //nodo interno
private:
    struct nodo{
        paciente p; // paciente con el nombre y la prioridad
        nodo * siguiente; // nodo siguiente
        //parte constructora
        nodo (const paciente& paciente) : p(paciente),
siguiente(nullptr){}

    };
    nodo * cabeza; //esta parte es donde esta el mas urgente de la
lista
public:
    //constructor de la lista vacia
    colaprioridad() : cabeza(nullptr) {}

    //en esta parte sabemos si la lista esta vacia
    bool estavacia() const {
        return cabeza == nullptr;
    }

    // parte del enqueue de la cola de prioridad
    void enqueue(const string& nombre, int prioridad) {
        //1. parte para validar la prioridad
        if(prioridad < 1 || prioridad > 4 ){
            cout<<" prioridad incorrecta: debe de ser del 1 al 4";
        }
    }

```

```

//2. construccion del paquete
paciente paciente{ nombre, prioridad};

//3. creamos un nuevo nodo
nodo * nuevo =new nodo(paciente);

//4. caso especial donde si la lista esta vacia o el nuevo que
ingrese es mas urgente que los otros entra al inicio
if (estavacia() || prioridad < cabeza -> p.prioridad){
    nuevo -> siguiente = cabeza; // el nuevo puntero apunta a
la anterior cabeza
    cabeza = nuevo;
    return;
}

//en esta parte buscamos donde insertar cuando el de la
siguiente prioridad sea menor o igual al nuevo
nodo * actual = cabeza;
while(actual->siguiente != nullptr && actual -> siguiente ->
p.prioridad <= prioridad){
    actual = actual -> siguiente; //aqui avanzamos
}

//6. nuevo apunta al siguiente de actual
nuevo -> siguiente = actual ->siguiente;
actual -> siguiente = nuevo; // actual apunta a nuevo

}

//parte del dequeue
paciente dequeue(){
    //1. validemos que no este vacia
    if( estavacia()){
        cout<< " cola de priorirdad esta vacia, por lo que no se
puede hacer dequeue";
    }
    //2. guardamos cabeza en temporal
    nodo * temp = cabeza;

    //3. guardamos el paciente de salida
    paciente salida = cabeza -> p;

```

```

        //4. mover cabeza al siguiente
        cabeza = cabeza ->siguiente;

        //5. liberamos memoria del nodo que quitamos
        delete temp;

        //6. retornamos al paciente
        return salida;

    }

    //metodo imprimir
    void imprimir() const{
        nodo * actual = cabeza; //empezamos en cabeza
        while (actual != nullptr){
            cout<< "p" << actual -> p.prioridad << "-" << actual ->
p.nombre;
            if (actual -> siguiente) cout <<" -> "; //flecha si es que
hay siguiente
            actual = actual -> siguiente; //avanza
        }

        cout << endl;
    }
};

int main(){
    //===== pruebas en main =====
    cout << "===== Recepcion =====" << endl;

    colaFIFO recepcion;

    //parte para probar estavacia()
    cout << "estavacia (esperando 1): " << recepcion.estavacia() << endl;

    //parte para probar enqueue
    recepcion.enqueue({"ana", 3});
    recepcion.enqueue({"jose", 1});
    cout << "verfrente (esperando ana): " <<recepcion.verfrente().nombre <<
endl;

    //parte para probar dequeue

```

```

paciente p1 = recepcion.dequeue();
cout << "dequeue (esperando ana): " << p1.nombre << endl;
cout << "nuevo frente (esperando jose): " <<
recepcion.verfrente().nombre << endl;

cout << "==== Cola de prioridad =====> <<endl;
// parte para probar la cola de prioridad
colaprioridad triage;

//insertar a 3 pacientes
triage.enqueue("Ana", 3);
cout << "Despues de P3-Ana: ";
triage.imprimir(); // esperado: P3-Ana

triage.enqueue("Jose", 1);
cout << "Despues de P1-Jose: ";
triage.imprimir(); // esperado: P1-Jose -> P3-Ana

triage.enqueue("Maria", 2);
cout << "Despues de P2-Maria: ";
triage.imprimir(); // esperado: P1-Jose -> P2-Maria -> P3-Ana

return 0;
}

```

3.- Cola Circular con Arreglo — Implementación con Módulo

```

#include <iostream>
using namespace std;
#include <string>
#include <stdexcept>

class colacircular{
private:
static const int capacidad = 5; // se define la capacidad y al ser
static y const, se conoce el tiempo de compilacion lo que permite usar
string
string dato[capacidad];

int frente;
int final;

```



```

int tamano;

public:
//parte constructora
colacircular():
frente(0), // iniciializamos frente y final en 0
final(0),
tamano(0) // tamaño es igual a cero ya que no hay valores
{}

bool estavacia() const { // estavacia devuelve true si no hay elementos
    return tamano == 0;
}

bool estallena() const{ //estallena devuelve true si ya no cabe nada
mas
    return tamano == capacidad;
}

void encolar(const string& x){

    //encolar agrega un valor a la cola
    if(estallena()){
        throw runtime_error("La cola esta llena"); // esto es para cuando
este llena la cola no se pueda insertar mas
    }
    //guardamos el elemento en la posicion final (que es la siguiente
casilla libre)
    dato[final] = x;
    final = (final + 1) % capacidad; // esto sirve para mover una
casilla adelante de forma circular
    tamano++;
}

string desencolar(){
    if(estavacia()){ // esto es por si la cola esta vacia
        throw runtime_error("La cola esta vacia");
    }
    string x = dato[frente]; //tomamos el elemento que esta en frente
que es el que sale
    frente = (frente + 1) % capacidad; // movemos frente una caslla
adelante de forma circular
    tamano--;
}

```

```

        return x;
    }

    void imprimir(const string& titulo) const{
        cout << "\n== " << titulo << " ==\n";

        // mostramos los indices y el tamaño
        cout << "front=" << frente
                << ", rear=" << final
                << ", size=" << tamaño
                << " (CAP=" << capacidad << ")\n";

        // Mostramos el arreglo completo (solo 5 posiciones).

        //mostramos el arreglo completo
        cout << "arreglo (indices 0..4):\n";

        for (int i = 0; i < capacidad; i++) {
            // Si data[i] está vacío, imprimimos "_" para visualizar
            // mejor.
            cout << "  [" << i << "] = " << (dato[i].empty() ? "_" :
            dato[i]) << "\n";
        }
        //mostramos el orden logico de la cola
        cout << "Orden logico de salida (desde frente, hasta tamaño ): ";
        for (int k = 0; k < tamaño; k++) {
            int idx = (frente + k) % capacidad; //idx es el indice real en
            // el arreglo

            cout << dato[idx]; // imprimimos el elemento en el indice

            if (k + 1 < tamaño) cout << ", ";

        } cout<<endl;
    }

};

int main() {

```

```

    // Creamos una cola circular vacía.
    colacircular q;

    // 1) Enqueue: A, B, C, D, E (debe quedar llena)
    q.encolar("A");
    q.encolar("B");
    q.encolar("C");
    q.encolar("D");
    q.encolar("E");
    q.imprimir("Despues de enqueue A..E (debe estar llena)");

    // Dequeue: A, B
    // Sale A (front avanza a 1)
    string out1 = q.desencolar();
    // Sale B (front avanza a 2)
    string out2 = q.desencolar();

    // Imprimimos lo que salió para comprobar que fue A y B.
    cout << "\nDequeue -> " << out1 << ", " << out2 << " (debe ser A,
B)\n";
    q.imprimir("Despues de dequeue A, B");

    // Enqueue: F, G (debe reutilizar índices 0 y 1)
    // En este punto rear seguramente está en 0 (porque ya había dado
la vuelta al llenarse).
    // Como sacamos 2 elementos, hay espacio en dos casillas: 0 y 1
(las que "dejaron" A y B).
    q.encolar("F");
    q.encolar("G");
    q.imprimir("Despues de enqueue F, G (reutiliza 0 y 1)");

    // Verificar estado final

    cout << "\nVerificacion final esperada: C, D, E, F, G\n";

    return 0;
}

```

FASE 3: REFLEXIONA

4-. Análisis de Trade-offs y Decisiones de Diseño

La evidencia del análisis va dentro del mismo archivo de código del HITO 1, como un bloque de comentarios al final con el prefijo # REFLEXIONA:. Debe incluir tres respuestas propias:

REFLEXIONA-1: A partir de cuántos pacientes simultáneos cambiarías de lista ordenada a heap, con un número justificado.

REFLEXIONA-2: Cómo tu implementación maneja el empate de prioridad (¿sale Ana antes que María si ambas son P2?). Incluye el resultado de la prueba.

REFLEXIONA-3: Describe en 3-5 líneas el flujo de datos entre Cola, ColaPrioridad y Pila en el sistema.

No se requiere documento separado. Los comentarios en el código son la evidencia.

REFLEXIONA-1:

el punto de quiebre de cuando ocupas cambiar la estructura es cuando llegas aproximadamente a los 100k. si bien en este caso es difícil que lleguen 100k pacientes a un hospital es donde se siente la diferencia de rendimiento

REFLEXIONA-2:

mi implementación tiene un contador de turnos con la cual se puede ver quien llegó primero, así que aunque ambos pacientes hayan llegado en tiempos casi iguales, se puede ver quien llegó primero de esos dos

REFLEXIONA-3:

FASE 4: VALIDA

✓ Checklist de Autoevaluación Rápida — Verifica antes de entregar

- ✓ **¿enqueue en cola vacía funciona correctamente?** Prueba: crea una cola nueva, haz enqueue(X), verifica que frente y final apunten al mismo nodo.
→ Si falla: revisa el caso especial "cola vacía" en el método enqueue. frente y final deben inicializarse al mismo nodo nuevo.
- ✓ **¿dequeue deja la cola completamente limpia cuando se extrae el último elemento?** Prueba: enqueue(A), dequeue(), verifica que TANTO frente COMO final sean null.
→ Si solo frente es null pero final no: falta el caso especial en dequeue() cuando frente queda vacío. Este es el bug más común de la semana.
- ✓ **¿dequeue en cola vacía lanza error/excepción en lugar de colapsar silenciosamente?** Prueba: crea cola vacía, llama dequeue(), verifica que el mensaje de error sea claro.
→ Si el programa colapsa sin mensaje: falta la verificación estaVacia() al inicio de dequeue(). Los errores silenciosos son más peligrosos que los ruidosos.
- ✓ **¿La Cola de Prioridad extrae siempre el elemento de mayor prioridad (menor número)?** Prueba: enqueue(A, P3), enqueue(B, P1), enqueue(C, P2). El primero en salir debe ser B (P1), no A.
→ Si sale A primero: la inserción ordenada no está comparando prioridades correctamente, o está insertando al final en vez de en la posición correcta.

- ✓ **¿Los empates de prioridad respetan el orden de llegada (FIFO dentro del mismo nivel)?** Prueba: enqueue(Ana, P2), enqueue(María, P2). Ana debe salir primero.
→ Si María sale primero: la inserción ordenada está usando estrictamente menor en lugar de menor o igual para saltar elementos del mismo nivel. Ajusta la condición de comparación.
- ✓ **¿El tamaño de la cola se actualiza correctamente con cada enqueue y dequeue?** Prueba: enqueue 3 veces, dequeue 1 vez. El tamaño debe ser 2, no 3 ni 1.
→ Si el tamaño no coincide: revisa que ambas operaciones incrementen/decrementen el contador. También verifica que no se incremente en el caso especial.

💡 Si encuentras un error con este checklist, dibuja el estado de **frente** y **final** antes y después de la operación problemática. Muchos bugs de colas se hacen evidentes cuando ves el diagrama de punteros.